

练习题（十四）

专题：布尔表达式与控制流语句的翻译（编译原理）

一、单选题

1. 关于布尔表达式的叙述，不正确的是（）。

- A. 布尔常量是布尔表达式
- B. 布尔常量不是布尔表达式
- C. 关系表达式是布尔表达式
- D. 将括号和逻辑运算符作用于布尔表达式得到一个新的布尔表达式

【正确答案】 B

【解析】 根据布尔表达式的文法定义，布尔常量（如 `true` 和 `false`）以及形如 `a < b` 的关系表达式都是最基本的布尔表达式。通过逻辑运算符（如 `and`, `or`, `not`）和括号可以组合成更复杂的布尔表达式。因此B的叙述是不正确的。

2. 以下说法不正确的是（）。

- A. 为布尔表达式和控制流语句生成目标代码时，关键问题之一是确定跳转指令的目标标号
- B. 在生成跳转指令时，就可以确定目标标号
- C. 在生成跳转指令时，目标标号还不能确定
- D. 可以将标号的地址作为继承属性传递到生成相关跳转指令的地方，但是这样的做法需要再进行一趟处理，将标号和具体地址绑定起来

【正确答案】 B

【解析】 在中间代码或目标代码生成的过程中，很多时候编译程序扫描到并需要生成一个跳转指令时，该跳转指令所要去的“目标地址（标号）”在当前的代码流中尚未被扫描到（即属于向后跳转），因此目标标号在此时是**无法立即确定**的。这也就是为什么在实际编译实现中需要引入“拉链与回填（Backpatching）”技术，或者采用D选项所说的多遍（多趟）处理。因此选项B的叙述是不正确的。

3. 令非终结符B表示布尔表达式，为B设置两个继承属性true和false。B.true是一个地址，该地址中存放了当B为真时控制流转向的指令的标号；B.false也是一个地址，该地址中存放了当B为假时控制流转向的指令的标号，则下列SDT中不正确的是（）。【布尔表达式的SDT】

- A. $B \rightarrow true \{ \text{gen('goto ' || B.true); } \}$
- B. $B \rightarrow false \{ \text{gen('goto ' || B.false); } \}$
- C. $B \rightarrow (B_1) \{ B_1.true = B.true; B_1.false = B.false; \}$
- D. $B \rightarrow not B_1 \{ B_1.true = B.true; B_1.false = B.false; \} B_1$

【正确答案】 D

【解析】 对于逻辑非（not）运算，其控制流逻辑应该完全相反：当整个表达式 B 为真时，子表达式 B_1 应该为假；当 B 为假时， B_1 应该为真。因此，正确的属性传递动作应该将真假出口互换，即： $B_1.true = B.false; B_1.false = B.true;$ 。选项D中直接透传了属性，没有进行互换，存在原则性逻辑错误。

4. 令非终结符B表示布尔表达式，为B设置两个继承属性true和false。B.true是一个地址，该地址中存放了当B为真时控制流转向的指令的标号；B.false也是一个地址，该地址中存放了当B为假时控制流转向的指令的标号，则下列SDT中不正确的是（）。【布尔表达式的SDT】

- A. $B \rightarrow true \{ \text{gen('goto ' || B.true); } \}$
- B. $B \rightarrow false \{ \text{gen('goto ' || B.false); } \}$
- C. $B \rightarrow (B_1 \{ B_1.true = B.true; B_1.false = B.false; \})$
- D. $B \rightarrow not \{ B_1.true = B.false; B_1.false = B.true; \} B_1$

【正确答案】 C

【解析】 本题的D选项修正了上题的逻辑错误，正确进行了真假真值的对调。我们需要关注语义动作在产生式右部中的**位置**。因为 `true` 和 `false` 是非终结符 B_1 的**继承属性**，根据L-属性文法和自顶向下翻译的原则，继承属性的计算动作必须放置在非终结符 B_1 的**左侧**（即在进入该节点前完成计算）。在C选项中，语义动作被误放在了 B_1 的**右侧**（在右部符号串中为 $B \rightarrow (B_1 \{ \dots \})$ ），这使得在遍历 B_1 时其继承属性尚未被赋值，因而是错误的。

二、判断题

5. 在分支和循环中会用到条件式，而用作条件式的通常是布尔表达式。 ()

【判断结果】√（正确）

【解析】在高级程序设计语言的控制流语句中（如 `if (condition)` 或 `while (condition)`），其中充当诸如条件判断、循环边界的条件式（`condition`），在绝大多数语言定义中都是由布尔表达式（包含关系表达式与逻辑表达式）来担任的。

6. 在控制流语句的翻译中，布尔表达式B被翻译成由跳转指令构成的跳转代码。 ()

【判断结果】√（正确）

【解析】在采用短路求值（Short-circuit evaluation）的控制流翻译方法中，布尔表达式不需要也不用在运行时真正计算出其具体的0或1的代数真值，而是根据各项的真假直接翻译成由条件跳转（如 `if a < b goto L1`）与无条件跳转组成的“跳转代码序列”。

7. 在跳转代码中，逻辑运算符&&、||和!被翻译成跳转指令。 ()

【判断结果】√（正确）

【解析】采用短路求值法时，高级语言中的逻辑运算符本身并不对应目标代码中具体的逻辑位运算指令，而是直接转化为控制流的跳转。例如 `B1 && B2` 在翻译时，若 `B1` 为假则控制流直接跳转到整个表达式的 `false` 出口，这种逻辑关系的串并联完全是通过跳转指令的结构来实现的。

8. 逻辑运算符&&、||和!会出现在代码中。 ()

【判断结果】×（错误）

【解析】此处的“代码”指的是编译生成的底层代码（如中间代码/三地址码/目标指令）。如前所述，在控制流代码生成的最终结果中，高级语言里的逻辑运算符 `&&`、`||` 和 `!` 已全部被“消去”，完全被转化为由关系运算符（如 `<`，`>`）和条件/无条件跳转指令构成的指令序列。因此这些高级逻辑运算符符号本身**不会**作为操作码出现在最终生成的代码中。